

# Technical Specification

## HYBRID RECOMMENDATION ENGINE V2.0

This document outlines the architecture, data schema, and mathematical logic required to upgrade the newsfeed recommendation engine. The objective is to transition from a literal, exact-keyword matching system to a semantic, weighted-score engine. This will dynamically balance explicit user settings with implicit feedback (Green/Amber/Red ratings) while mitigating the risk of filter bubbles.

### Phase 1: Data Architecture & The Semantic Layer

Pure keyword matching fails due to a lack of context. The ingestion pipeline must extract structured meaning from article text.

#### 1. Article Processing Pipeline

When a new article is ingested, it must pass through an NLP (Natural Language Processing) layer prior to database insertion.

- **Named Entity Recognition (NER):** Extract and log specific entities (People, Organizations, Locations, Events).
- **Topic Classification:** Assign broad taxonomic categories (e.g., *Technology*, *Politics*, *Sports*).
- **Actionable Task:** Implement an NLP pipeline (e.g., spaCy, HuggingFace, or Google Cloud Natural Language API) in the background worker processing the articles.

#### 2. User Profile Schema Updates

The user model must track both long-term explicit settings and short-term implicit interactions.

| Metric             | Source          | Data Type            | Purpose                                                                                          |
|--------------------|-----------------|----------------------|--------------------------------------------------------------------------------------------------|
| Explicit Interests | User Settings   | Array of Topic IDs   | Establishes the baseline positive score.                                                         |
| Explicit Dislikes  | User Settings   | Array of Topic IDs   | Acts as a hard veto (score = 0).                                                                 |
| Entity Affinity    | Article Ratings | Key-Value Dictionary | Tracks dynamic scores for specific entities over time (e.g., {"Elon Musk": 15, "Bitcoin": -10}). |

## Phase 2: The Scoring Algorithm

Every candidate article must be scored against the specific user requesting the feed. The total relevance score determines rank.

If article contains an Explicit Dislike:  $S_{total} = 0$

Otherwise:  $S_{total} = [ (W_{base} \times S_{base}) + (W_{dyn} \times S_{dyn}) ] \times e^{-\lambda t}$

### Variable Definitions

- $S_{base}$  (**Explicit Baseline Score**): If the article's broad category matches a user's Explicit Interest in settings, assign a baseline score (e.g., 50 points).
- $S_{dyn}$  (**Dynamic Entity Score**): The sum of the user's affinity scores for the specific entities found in the article.
- $W_{base}$  and  $W_{dyn}$  (**Weights**): Tuning parameters (floats) determining the balance between implicit ratings and explicit settings. (Suggested starting values:  $W_{base} = 0.4$ ,  $W_{dyn} = 0.6$ ).
- $e^{-\lambda t}$  (**Time Decay Factor**): Ratings age over time.  $t$  is the age of the rating in days, and  $\lambda$  controls the rate of decay.

### Implicit Feedback Adjustments

When a user rates an article, update their "Entity Affinity" dictionary for the entities found in that specific article:

- **GREEN** (**Very Interested**): Add **+10 points** to the extracted entities.
- **AMBER** (**Mildly Interested**): Add **+2 points** to the entities. Signals acceptable filler content.
- **RED** (**Not Interested**): Subtract **-15 points** from the entities.

**Crucial Red Logic:** Do not penalize the broad topic (e.g., "Politics"), only the specific entities (e.g., "Senator John Doe"). This prevents a single bad article from wiping out an entire category of interest.

## Phase 3: Feed Construction & Mixing

Sorting the database purely by  $S_{total}$  highest-to-lowest creates a highly restricted filter bubble. To ensure a "decent selection," the feed compilation endpoint should paginate articles in batches of 10, using the following distribution:

| Segment                            | Percentage    | Selection Logic                                                                                                 |
|------------------------------------|---------------|-----------------------------------------------------------------------------------------------------------------|
| <b>The Core (Exploitation)</b>     | 70% (7 items) | Top-ranked articles based on highest $S_{total}$ .                                                              |
| <b>The Fringe (Exploration)</b>    | 20% (2 items) | Articles driven primarily by Amber ratings, or secondary entities with low but positive $S_{dyn}$ scores.       |
| <b>The Wildcards (Serendipity)</b> | 10% (1 item)  | High-engagement articles globally, completely ignoring the user's profile (unless it hits an Explicit Dislike). |

## Phase 4: Cold Start & Optimization (Further Recommendations)

### The "Cold Start" Problem

When a new user registers, they have no explicit settings or implicit ratings. To prevent an empty or irrelevant initial feed:

- **Global Trending Baseline:** Default the user's feed to the platform's top-performing articles (measured by overall Green ratings from the general user base) grouped by diverse categories.
- **Onboarding Wizard:** Force a lightweight selection of 3-5 broad topics during registration to immediately seed the  $S_{base}$ .

### Database Performance

Calculating  $S_{total}$  for every article in the database per user on every refresh will cause severe latency. The developer must implement a tiered filtering approach:

1. **Time-Boundary Pre-filter:** Query only articles published within the last 48-72 hours.
2. **Hard Exclusions:** Immediately drop articles containing tags matching the user's Explicit Dislikes.
3. **Score Computation:** Run the  $S_{total}$  math only on this reduced subset of articles.
4. **Caching:** Cache the user's generated feed and only recompute when they exhaust the current batch or force a manual refresh.